

— BACKEND ENGINEER · CLOUD NATIVE SYSTEMS

Seunghwan Seol

EVIDENCE-DRIVEN BACKEND ENGINEERING

AI를 적극 활용하되, 결과를 그대로 받아들이지 않고 실측하며 검증하고 판단합니다.

한 번의 결과에 의존하지 않고 반복 측정으로 오차를 줄이며, AI가 제시한 해결안도 근거를 확인한 뒤 직접 결정합니다.

탐색과 개발의 속도는 높이되 판단의 책임은 맡기지 않고, 신뢰할 수 있는 결과를 만드는 개발자가 되고자 합니다.

✉ tjftmd1@naver.com github.com/SH-Seol velog.io/@tjftmd1 🎓 경희대 컴퓨터공학과 (2025.08 졸업)



판단과 검증의 기준

🔍 AI는 탐색을 빠르게 하는 도구

Codex와 Claude를 활용해 선택지를 넓히되, 제안된 해결안을 정답으로 취급하지 않습니다. 근거와 제약을 확인한 뒤 직접 판단합니다.

📊 실측으로 검증하는 습관

성능과 설계의 결론을 추측으로 마무리하지 않습니다. 반복 측정과 관측 지표를 통해 결과를 확인하고, 오차와 실패 원인을 다시 검토합니다.

👉 설명 가능한 의사결정

선택의 이유와 대안을 동료가 이해할 수 있게 설명합니다. 판단과 결과를 신뢰할 수 있도록, 과정과 근거를 남기는 협업을 지향합니다.

Certification & Education

- CKAD · Certified Kubernetes Application Developer (2026.07)
- 정보처리기사 (2025.09)
- AWS Certified Developer – Associate (2025.06)
- SQLD (2025.04)
- OPIc 영어 IH (2026.09)

경희대학교 컴퓨터공학과 학사 · 2019.03 – 2025.08

LG U+ URECA Backend Training · 2026.04 – 2026.09

프로젝트 개요

개인 프로젝트 · OBSERVABILITY PLATFORM

IN PROGRESS

Observation

2026.09 - 현재

앱 소스·빌드 수정 없이 텔레메트리를 연결하는 조사 플랫폼. OTLP 수집 경로와 내부 API로 서비스 → route → trace → 상관 로그를 연결.

OpenTelemetry

Prometheus · Tempo · Loki

PostgreSQL

팀 프로젝트 · BACKEND + CLOUD INFRASTRUCTURE

IN PROGRESS

Sapari

2026.04 - 현재

실시간 영상 스트리밍 기반 라이브 커머스 플랫폼. LiveKit 기반 스트리밍 파이프라인과 함께 중복 시작을 막고, 비정상 종료 뒤 남는 orphan live-미디어 리소스를 회수하도록 설계.

AWS · Amazon EKS

Kafka

LiveKit WebRTC

Terraform

S3

팀 프로젝트 · BACKEND + PERFORMANCE

Coupon Yaho

2026.08 - 2026.09

선착순 쿠폰 발급 시스템. 재고 1만장·총 요청 2만건·500 req/s 조건에서 발급 경로와 관제 체계를 개선해 발급 p99 68%, 매진 거절 p99 99% 개선.

Spring Boot

Redis Lua

Kafka

Prometheus

k6

개인 프로젝트 · BACKEND

Medicall

2025.06 - 2026.01

일차의료 방문진료 수가 시범사업 기반 의료 플랫폼. Spring Boot 멀티모듈 아키텍처로 의료 워크플로우 전반 구현. AI 진료 요약, 실시간 채팅, 처방전 QR까지.

Java 17

Spring Boot 3.5

OpenAI API

Redis

QueryDSL

팀 프로젝트 · BACKEND (UMC 6기)

YelloBook

2024.06 - 2024.11

소상공인 업무·재고 관리 SaaS 백엔드. 전체 880커밋 중 227건(26%) 기여. Team·Inform 두 도메인 풀스택 단독 담당 + 테스트 인프라 설계.

Java 17

Testcontainers

DCI 패턴

QueryDSL

Redis

팀 연구 · PROTOCOL ANALYSIS

WebTransport vs WebSocket

2024.03 - 2024.06

HTTP/3·QUIC 기반 WebTransport의 실시간 스트리밍 성능 정량 검증. Jitter 38% 개선 확인. 30만/100만 건 데이터셋 성능 비교 실험.

Golang

QUIC

WebTransport

Wireshark

PROJECT 01 · PERSONAL PROJECT · 2026.09 - PRESENT

Observation.

TELEMETRY COLLECTION · INCIDENT INVESTIGATION PLATFORM

애플리케이션 소스와 빌드를 수정하지 않고 텔레메트리를 연결하고, 메트릭·트레이스·로그·배포 이력을 한 조사 흐름으로 잇는 로컬 관측 플랫폼입니다.

OpenTelemetry Java Agent

OTLP push

Prometheus · Tempo · Loki

PostgreSQL

다음: 수집 경로와 내부 조사 API →

수집에서 조사까지, 경로를 하나로 만들었습니다

앱을 Prometheus scrape 대상으로 노출하지 않고 OTLP push를 단일 입력으로
사용했습니다. 실제 연동 범위는 HTTP·JDBC 텔레메트리입니다.



빈 데이터도 상태로 만들고, 판정 불가를 성공에서 분리했습니다

관측 화면에 값이 없다는 사실만으로 서비스를 정상이라 판단하지 않도록, 데이터
상태와 검증 결과를 명시적으로 반환합니다.

7개 데이터 상태

정상·데이터 부재·플랫폼 장애를 같은 빈 화면으로 합치지 않습니다.

available 조회 가능한 데이터	no_data 조건에 맞는 데이터 없음
stale 수집 지연·오래된 데이터	unsupported 지원하지 않는 신호
temporarily_unavailable 저장소 일시 장애	misconfigured 연결·설정 오류
internal_error 관제 플랫폼 자체 오류 — 다른 패널은 계속 응답	

검증도 결과의 일부

실행하지 못했거나 판정할 수 없는 검증을 통과로 기록하지 않습니다.

Contract	API 응답 형태와 상태 계약을 고정
Smoke	수집 경로별 최소 신호 도달 여부 확인
Scenario	저장소 장애·설정 오류 등 실패 조건 분리
CI gate	반복 가능한 규칙은 negative test와 종료 코드로 차단

측정 결과 · Agent 유무를 조건당 3회 교차 실행했습니다. 해당 조건에서 기동은 1.4→3.1초였고, 처리량·P99는 측정 범위가 겹쳐 차이를 단정하지 않았습니다.

PROJECT 02 · TEAM PROJECT · 2026.04 - PRESENT

Sapari.

LIVE COMMERCE PLATFORM · BACKEND & CLOUD INFRASTRUCTURE

실시간 영상 스트리밍을 중심으로, 중복 방송을 막고 비정상 종료 뒤 남는 orphan live·미디어 리소스를 회수하는 흐름까지 설계한 라이브 커머스 프로젝트입니다.

AWS · EKS design

Terraform

LiveKit · WebRTC

HLS streaming

다음: 아키텍처와 라이브 스트리밍 설계 →

EKS 기반 라이브 커머스 인프라

AWS · Amazon EKS · Terraform IaC
라이브·채팅 AWS 이전 설계

노드그룹을 미디어와 분리

egress가 4~6 vCPU를 지속 점유해도 앱을 밀어내지 않게 했습니다.

Terraform 스택을 분리

storage를 먼저 apply하고, 실측 뒤 노드 규모를 정합니다.

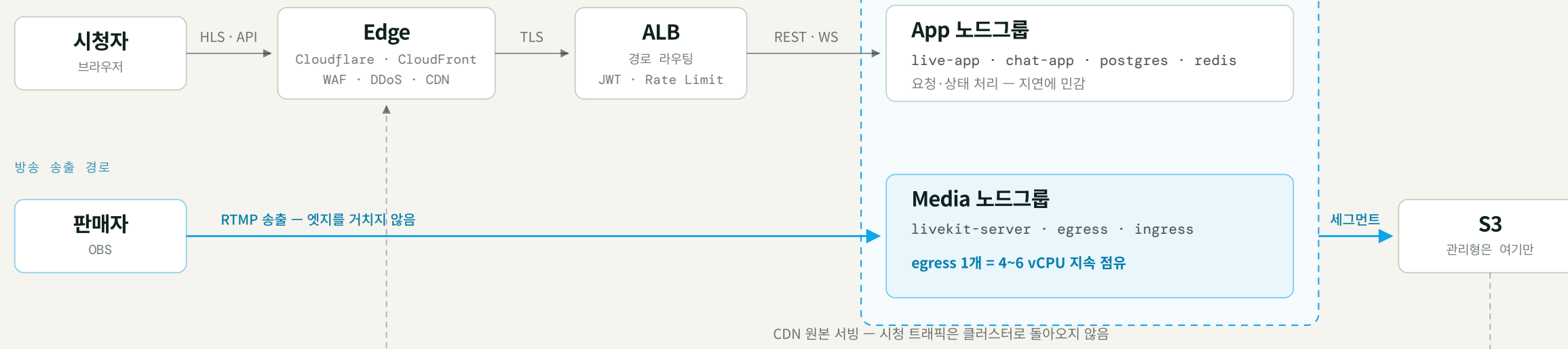
켜고 끄는 것을 전제로 설계

스택 단위 apply·destroy로 검증 비용을 통제합니다.

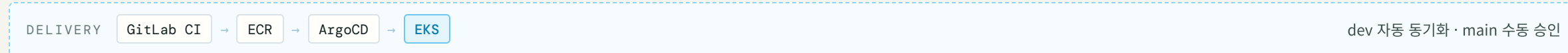
관리형 대신 클러스터 내부

PostgreSQL·Redis는 파드, S3만 관리형으로 둡니다.

시청 · API 경로



시청 경로와 송출 경로가 클러스터 안에서 서로 다른 노드그룹으로 갈라집니다. 세그먼트는 S3로 빠져 CDN이 대신 서빙합니다.



도메인은 LiveKit을 모릅니다

미디어 서버는 언제든 갈아끼울 수 있어야 했습니다. 그래서 **도메인이 LiveKit 타입을 직접 참조하지 못하게** 막고, 관심사마다 실행 진입점을 나눴습니다.

REPOSITORY LAYOUT

```
sapari-be/
├── apps/                # 실행 진입점 - 배포 단위
│   ├── api             REST API      · App 노드그룹
│   ├── admin           Admin         · App 노드그룹
│   ├── streaming       Streaming     · Media 노드그룹
│   └── batch            Batch         · App 노드그룹
├── modules/            # 도메인 - 바깥을 모른다
│   ├── user · product · live · order
│   └── promotion · notification
├── storage/            # 어댑터 - JPA · Redis · LiveKit · S3
└── commons/            # 도메인 지식 없는 공통 유틸만
```

DEPENDENCY RULES

01 modules/ 는 storage/ 를 참조하지 않습니다

도메인은 포트 인터페이스만 알고, 구현은 바깥에서 주입받습니다. LiveKit 응답 타입이 도메인까지 올라오면 미디어 서버 교체가 전 계층 수정이 됩니다.

02 apps/ 끼리는 서로를 모릅니다

진입점은 각자 뜨고 각자 죽습니다. Streaming만 Media 노드그룹에 배포하면 되고, 미디어 때문에 API까지 같이 스케일할 이유가 없습니다.

03 commons/ 는 얇게 유지합니다

공통이 커지면 모든 모듈이 그걸 통해 간접적으로 연결됩니다. 두 곳 이상에서 쓰이고 도메인 지식이 없는 것만 남깁니다.

이 규칙을 사람이 지키지 않게

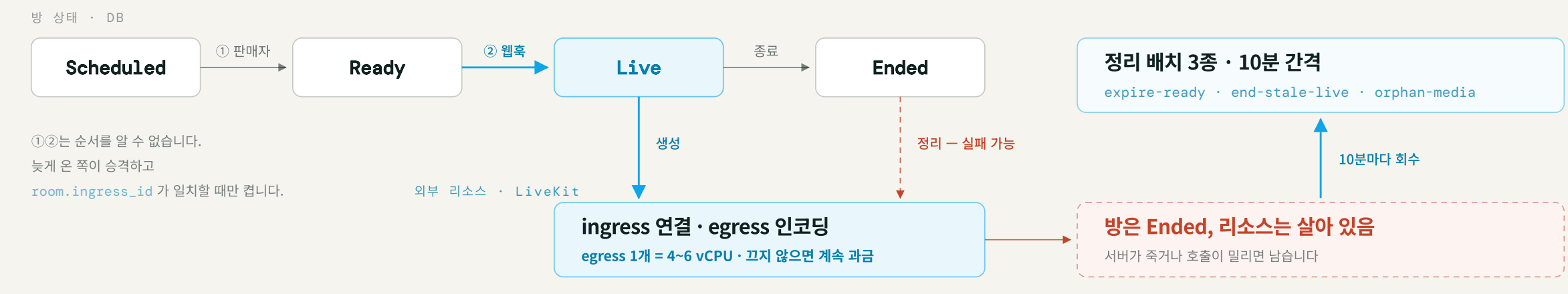
```
noClasses().that().resideInAPackage("..modules..")
    .should().dependOnClassesThat()
    .resideInAPackage("..storage..")
```

모듈 경계를 ArchUnit 테스트로 고정해, 위반하면 빌드가 깨집니다. 규칙을 문서로 두면 리뷰에서 놓치지만 테스트로 두면 놓칠 수 없습니다.

방을 끄는 것보다 안 꺼지는 걸 찾는 게 어렵습니다

시작 경합 · 리소스 회수
정리 배치 3종 · 10분 간격

컬 때는 신호가 두 갈래로 오고, 끌 때는 정리 호출이 실패할 수 있습니다. 둘 다 방 상태와 LiveKit에 실제로 존재하는 리소스가 어긋나는 문제입니다.



①②는 순서를 알 수 없습니다.
늦게 온 쪽이 승격하고
room.ingress_id 가 일치할 때만 컵니다.

방 상태와 외부 리소스는 따로 삽니다. 어긋나는 두 지점 — 시작 경합과 정리 실패 — 을 각각 승격 조건과 회수 배치로 막았습니다.

실패 방향

틀렸을 때 어디로 넘어질지 정했습니다

정리 호출은 실패해도 넘어갑니다 — 배치가 치웁니다. 반대로 전체 목록 조회는 200 + []도 실패로 봅니다. 믿으면 방송 중인 방을 끕니다.

순서

정리는 커밋 뒤로 뺐습니다

종료를 저장하는 트랜잭션 안에서 정리까지 하면, 정리가 실패했을 때 종료까지 되돌아갑니다. 판매자는 컸는데 방은 살아 있는 셈입니다.

관측

실패율 0이 거짓이었습니다

배치 실패율이 0으로 나왔는데, 정리 계열이 실패를 삼키고 정상 반환해 실패가 지표까지 도달할 수 없는 구조였습니다. 읽는 법을 주석과 테스트에 남겼습니다.

도메인 지식을 바탕으로 AI를 활용합니다

2026.06 – 09 · 세션 48회
설계 대안 탐색에 AI · 판정은 도메인으로

HOW I USED AI

넓게 받습니다

설계 대안과 구현 초안은 AI로 빠르게 펼칩니다.
여기까지는 전부 후보입니다

상황을 대입합니다

코드가 맞는지 보기 전에 실제 판매자·장비·방송
흐름에 넣어 반례를 찾습니다

통과한 것만 남깁니다

반례를 건넌 설계만 채택하고, 그 이유를 주석·
테스트로 고정합니다

2026.06.28 · 송출 방식 선택

“실제 판매자들은 자기가 RTMP인지 WebRTC인지 판단을 못 할 것 같거든?”

입력값으로 송출 방식을 받는 설계였습니다. 카메라 여러 대에 DSLR을 쓰는 판매자는 **WebRTC로 장비가
불지 않습니다** — 고를 수 있는 선택지가 아니었습니다.

설계 변경

플래그 대신 **경로별 엔드포인트**로 분리
PrepareIngress · GoLiveByRtmp · StartLive

2026.06.24 · Egress 일괄 중단

“**isStoppable**에 **EGRESS_ENDING**은 왜 있는 거지? ending이면 끝난 상태 아니야?
stoppable이 아니지 않아?”

이미 멈추는 중인 작업에 중단을 또 보내는 조건이었습니다. **상태 이름이 아니라 그 상태에서 무엇이 남아
있는지**로 판정해야 했습니다.

조건 축소

중단 대상을 STARTING·ACTIVE로 한정
제외 근거를 **주석으로 고정**

2026.07.31 · orphan room 만료

“확인하고 아니라고 판단한 뒤, 작업하는 사이에 LiveKit이 live로 바꿨다면 결국 같은 거
아니야?”

검사와 조치 사이가 열려 있었습니다. **검사 시점과 사용 시점이 다르면 그 판정은 근거가 되지 못합니다.**

경합 차단

행 잠금으로 전이를 직렬화
순서 복원 금지를 주석에 명시

PROJECT 03 · TEAM PROJECT · 2026.08 - 2026.09

Coupon *Yaho.*

FLASH COUPON ISSUANCE · CONCURRENCY & OBSERVABILITY

재고 한 행에 수만 요청이 동시에 도착하는 선착순 발급.

“돌아가게 만드는 것”이 아니라 어느 부하에서 어떻게 무너지는지를 숫자로 아는 것을 목표로 잡았습니다.

AI 사용 원칙

발급 경로 개선

정합성 · 관제

다음: AI를 어떻게 썼나 →

설계는 맡기고, 채택은 맡기지 않았습니다

두 번 모두 테스트는 통과했습니다

CASE 1 · 발급 엔진

맡김 설계안·초기 구현. 잠금 전략과 트랜잭션 구조 선택지를 여러 벌 확보

보류 채택. 테스트 통과·과발급 0이었지만 **목표 부하에서 무너졌습니다**

결정 직접 읽고 **재고 잠금 구간·커밋 구조**를 원인으로 특정. 이후 구조 변경은 측정 근거 확인 후 채택

“돌아간다”와 “견딘다”는 실행 전에는 구분되지 않았습니다.

CASE 2 · REDIS 선점 구조

맡김 설계 검토와 보완 장치 후보 도출

보류 안전 판단. 제안 중 일부가 **초과 발급 방향**의 위험을 갖고 있었습니다

결정 거절하되 **재검토 조건**을 함께 기록. “틀렸다”가 아니라 “지금은 아니다”

되돌릴 수 없는 방향의 제안은 그럴듯할수록 위험합니다.

판단 기준 초과 발급은 되돌릴 수 없고, 재고 과소 표시는 되돌릴 수 있습니다. 이 저울 하나로 모든 결정을 정렬했습니다.

거절도 같은 줄에 서 있었습니다

재고 1만 · 요청 2만 · 500 req/s · 각 7회

요청 2만건 중 절반은 반드시 매진 거절인데, 그 1만 건도 **행 잠금을 기다려 INSERT 두 건을 실행한 뒤** 롤백되고 있었습니다.
세 지점의 트레이드오프를 따져 **무엇을 포기하고 무엇을 얻을지**를 먼저 정한 뒤 고쳤습니다.

01 · 잠금 위치

선행 잠금 vs 제약 위임

앞에서 잠그면 **코드 생성까지 임계구간에** 들어갑니다. 같은 배제를 `uk_coupon_member` 와 조건부 UPDATE가 이미 제공하고 있었습니다.

포기 제어 흐름의 단순함

획득 임계구간 밖으로 나온 코드 생성

02 · 락등 방식

선점 가시성 vs 단일 커밋

빈 행을 먼저 커밋하면 **커밋이 두 번**, 실패 정리 장치가 따라붙습니다. 유니크 제약이 같은 보장을 줍니다.

포기 진행 중 상태의 외부 관측

획득 커밋 3→1 · INSERT 7행→3행

03 · 진실의 위치

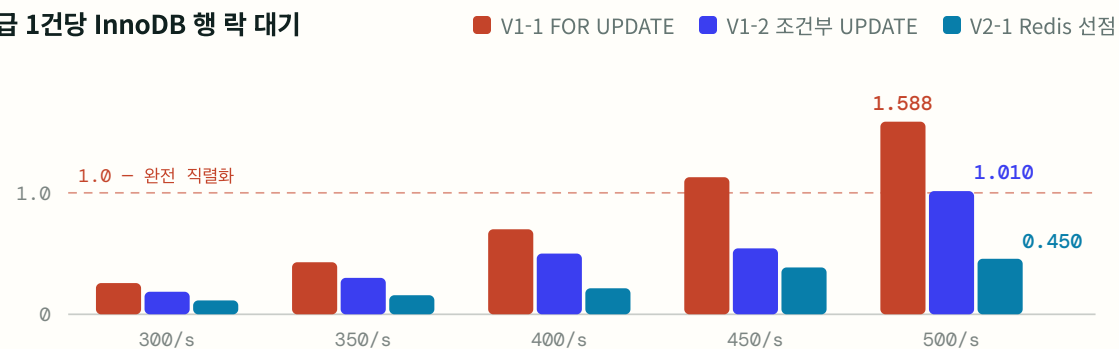
DB 단일 진실 vs Redis 선행 게이트

Redis로 옮기면 **맞출 속이 하나 늘어납니다**. 거절이 DB를 타지 않는 값이 더 컸습니다.

포기 단일 저장소 정합성

획득 거절 1왕복 종료 · 다음 장의 감시 측 4

발급 1건당 InnoDB 행 락 대기



임계를 넘는 도착률이 **400~450 → 500 req/s 초과**로 밀렸습니다. 500 req/s에서 응답 붕괴 **4/7회 → 0회**.

왜 p99가 아니라 락 대기인가

성공 p99는 도착률에 따라 **뒤집혀** 교차점을 잡을 수 없었습니다.

락 대기 **1.0 = 발급마다 평균 1회 이상 대기 = 완전 직렬화**. 자의적 임계가 아니라 물리적 기준입니다.

발급 성공 P99

2,240 → 716_{ms}

-68% · 500 req/s

매진 거절 P99

2,176 → 12.4_{ms}

-99% · 500 req/s

정합성은 감시가 아니라 구조로 지켰습니다

계측 4축 · 감시 축 4 · 과발급 0건

구조

자동 회수 스위퍼를 넣지 않았습니다

회수자의 “DB에 없다” 확인과 원 요청 커밋 사이에 **관측 불가능한 창**이 있습니다. 그 창에서 회수가 성사되면 재고가 되살아나 초과 발급이 됩니다.

- 포기: 죽은 선점 재고의 회수
- 획득: 되돌릴 수 없는 방향의 사고 차단

순서

깨는 방향의 연산을 나중에

- 발급: Redis → DB
줄이는 방향, 게이트가 앞
- 복원: DB → Redis
늘리는 방향, 늦어도 안전

불변식을 깨는 쪽 연산이 항상 마지막에 오도록 순서를 고정했습니다.

표현

0이 두 뜻을 갖지 않게

- 판정 없는 실행: NaN
- 빈 관측 창: 빈 값
- 미매핑 실패 코드: UNMAPPED

“0이다”와 “모른다”가 섞이면 없는 사고를 만들거나 있는 사고를 지웁니다.

계측 4축

무엇을 재는가

Latency

발급·거절 응답 분포

Traffic

도착률과 처리량

Error

실패 코드별 분류

Saturation

커넥션·락 대기 포화

감시 축 4

깨지면 사고 직전

LUA_GAP

Lua 원자성 파손

PERSIST_GAP

선점됐는데 DB에 없음

ACTIVE_DB_GAP

양수면 초과 발급 직전

DB_COUNTER_GAP

DB 내부 불변식 위반

Prometheus 1초 스크레이프 · 관리자 대시보드 API 1초 폴링

관제도 검증했습니다 알림은 안 올리는 것과 사고가 없는 것이 구분되지 않습니다. 규칙이 실제로 로드되고 발화하는지까지 테스트로 확인했고, 결함을 일부러 심어 그 테스트가 잡아내는지도 검증했습니다.

PROJECT 04 · PERSONAL PROJECT · 2025.06 - 2026.01

Medicall.

TELEMEDICINE & APPOINTMENT PLATFORM · BACKEND ARCHITECTURE

일차의료 방문진료 수가 시범사업을 바탕으로 환자·의사·병원 사이의 예약과 진료 흐름을 다뤘습니다. 멀티모듈 구조와 인증·보안·실시간 기능을 구현한 개인 프로젝트입니다.

Spring Boot

Multi-module

OAuth2 · JWT

Redis

다음: 아키텍처와 핵심 기능 →

멀티모듈 아키텍처

개인 프로젝트 · 2025.06 – 2026.01

MODULE STRUCTURE

API Layer med icall - api - patient · doctor · hospital · chat

↓ depends on

Common med icall - api - common (JWT, OAuth2, Swagger)

↓

Core (Domain Layer) med icall - core — Domain Record + Repository Interface

↑ implements

Storage Layer med icall - storage — JPA Entity, QueryDSL, Redis

핵심: med icall - core는 순수 도메인 모델(Record 기반)과 Repository 인터페이스만 보유, JPA에 의존하지 않음. storage가 인터페이스를 구현하고 Entity ↔ Domain 매핑 담당. 공통 코드 **66% 감소** (3,000줄 → 1,000줄).

SERVICE 컴포넌트 설계

🎯 Service — 트랜잭션 + 오케스트레이션

트랜잭션 경계를 책임지고 Reader/Writer/Validator를 조율. 비즈니스 흐름의 진입점.

📖 Reader / 🍷 Writer — CQRS 지향

조회와 변경 책임을 완전 분리. @Transactional(readOnly=true)를 조회 메서드에 개별 적용.

✅ Validator — 비즈니스 규칙 검증

AppointmentValidator가 중복·충돌·과거시간·접근권한을 전담. Service는 조율만.

AppointmentService → Validator → Writer

각 도메인 서비스는 검증→쓰기를 위임하고 흐름만 조율합니다.

핵심 구현 기능

6 Core Features



AI 진료 상담 요약

채팅 로그를 [D]/[P] 태그로 포매팅 후 OpenAI에 전송.
JSON Schema 출력 강제, 슬라이딩 윈도우(30 메시지)로
토큰 최적화. gpt-4o-mini 선택으로 비용 50% 절감.

- WebClient
- Non-blocking
- JSON Schema
- Sliding Window



OAuth2 + JWT + Redis 인증

소셜 로그인 → JWT 발급 → 쿠키 전달. Access/Refresh
분리, serviceType 클레임으로 사용자 유형 구분. Redis
블랙리스트로 즉시 로그아웃(토큰 무효화) 구현.

- OAuth2
- JWT Blacklist
- Redis
- OncePerRequestFilter



실시간 채팅 (커서 페이지네이션)

예약당 채팅방 getOrCreate로 멍등 생성. cursorId 기반
무한 스크롤, offset 성능 문제 회피. cursorId=0/null
엡지케이스 처리. 접근권한 검증.

- Cursor Pagination
- Idempotent
- ChatValidator



예약 비즈니스 규칙 검증

환자 동일시간 중복 차단, 의사 충돌 차단, 과거 시간 차단,
환자/의사/병원 접근권한 검증. AppointmentValidator
단독 책임.

- Domain Validation
- Conflict Detection



처방전 QR 코드 발급

ZXing으로 처방전 QR 토큰 URL 인코딩, Base64 PNG
반환. 약국에서 스캔으로 처방 확인 가능한 실용적 기능.

- ZXing
- Base64 PNG
- QR Token



Docker 멀티스테이지 빌드

빌드 스테이지와 런타임 스테이지를 분리해, 최종
이미지에는 실행에 필요한 JAR와 런타임만 남기도록
구성했습니다. 빌드 도구와 캐시를 배포 이미지에서
제외했습니다.

- Docker
- Multi-stage Build
- Runtime Image

PROJECT 05 · TEAM PROJECT · UMC 6TH · 2024.06 - 2024.11

YelloBook.

SMALL-BUSINESS WORK & INVENTORY MANAGEMENT SAAS · BACKEND

Redis를 활용한 초대 코드 흐름과, 데이터 규모별 @멘션 검색 방식을 직접 비교한 프로젝트입니다.

Redis TTL

UUID invitation

QueryDSL

Performance test

다음: Redis 설계와 @멘션 검색 성능 비교 →

Redis 기반 초대 코드와 @멘션 검색

30만 / 100만 건 데이터셋

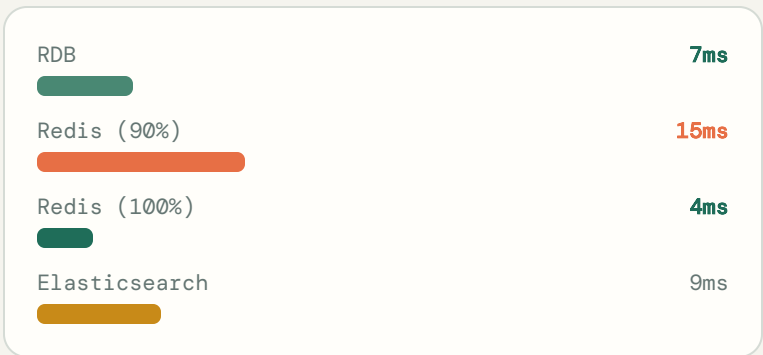
MY OWNERSHIP

Team · Inform 도메인 단독 담당

Testcontainers 기반 공통 RepositoryTest를 팀 표준으로 정착

227 commits · 전체 26%

30만 건 — AVG RESPONSE TIME



WHAT THE COMPARISON ESTABLISHED

EXPERIMENT DESIGN

데이터 규모와 캐시 적중률을 분리

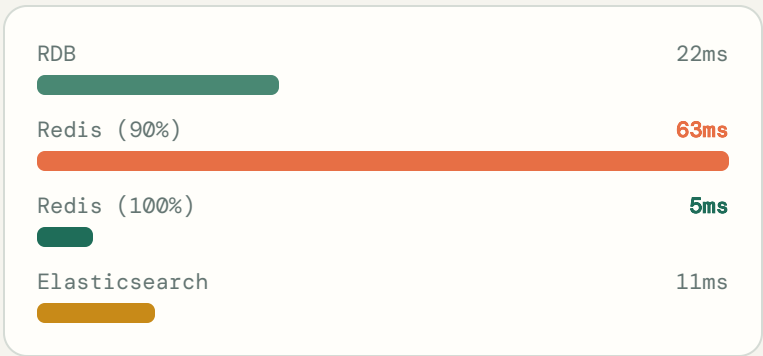
30만·100만 건에서 RDB, Redis(적중률 90%·100%), Elasticsearch를 같은 조건으로 비교하고 Avg·Max·P90을 함께 확인했습니다.

TRADE-OFF

Redis는 항상 빠르지 않았습니다

적중률 100%에서는 가장 빨랐지만, 90%에서는 캐시 조회·미스 처리 비용으로 100만 건 평균 63ms까지 느려졌습니다.

100만 건 — AVG RESPONSE TIME



READING THE RESULT

규모가 커질수록 안정성도 봤습니다

RDB는 100만 건에서 Max 739ms로 흔들렸고, Elasticsearch는 Avg 11ms·P90 22ms로 대용량에서도 비교적 안정적이었습니다.

DECISION CRITERIA

기술이 아니라 조건으로 선택

캐시 적중률·데이터 증가·응답 지연 허용 범위를 기준으로 경로를 선택해야 한다는 판단 기준을 만들었습니다.

UUID + Redis TTL 초대 코드

랜덤 UUID를 키로 발급하고 15분 TTL로 자동 만료되도록 설계했습니다. 별도 만료 처리 없이 초대 코드가 무효화되며, VIEWER·ADMIN 권한 분기에도 활용했습니다.

RESEARCH · TEAM STUDY · 2024.03 - 2024.06

WebTransport.

WEBTRANSPORT VS WEBSOCKET · REAL-TIME STREAMING PROTOCOL ANALYSIS

HTTP/3·QUIC 기반 WebTransport와 WebSocket을 비교하고, 네트워크 불안정 환경에서의 지연 변동성을 Wireshark로 측정한 연구입니다.

HTTP/3 · QUIC

Golang

Wireshark

Jitter analysis

다음: 실험 조건과 측정 결과 →

WebTransport vs WebSocket

2024.03 - 2024.06 · Golang

38%

Jitter 개선 (WebTransport)

≈

처리량·전송 시간 대등

JITTER 비교

WebSocket (TCP)

428.3μs

WebTransport (QUIC)

264.0μs

기술 과제 & 해결

브라우저 호환성 문제

고수준 라이브러리가 Go 1.18 비표준에 머물러 Chrome에서 미동작. quic-go 하위 패키지로 최신 드래프트 표준에 맞는 세션 업그레이드·단방향 스트림 로직 직접 구현.

스트리밍 최적화 전략

비디오를 프레임 단위 분할·독립 인코딩 전송. 일부 손실에도 전체 품질 유지. Wireshark 패킷 분석으로 Jitter 수치 정량화.

프로토콜 비교

특성	WEBSOCKET	WEBTRANSPORT
전송 계층	TCP	QUIC (UDP)
Jitter	428.3μs	264.0μs (▼38%)
TLS	선택적	강제 (보안 ↑)
HOL Blocking	있음	없음 (스트림 독립)
재전송 복구	느림	빠름 (QUIC 기반)
처리량	≈ 871k Bps	≈ 871k Bps (대등)

결론: 네트워크 불안정 환경의 실시간 스트리밍에서 WebTransport가 Jitter 38% 우위. 고도 보안 서비스 도입 타당성 확인. QUIC의 빠른 재전송·복구가 지연 변동성 개선에 직접 기여.

전송 효율

평균 전송 시간 ~3.16s, 처리량 ~871k Bps에서 두 프로토콜 대등. 실무 투입 가능 수준 확인.

보안 우위

WebTransport는 TLS 강제 적용으로 데이터 기밀성·무결성 기본 보장.

— CLOSING

문제의 원인을 파악하고,
해결 방안의 트레이드오프도 고려해
더 나은 시스템을 만들겠습니다.